

TEHNIČKA ŠKOLA RUĐERA BOŠKOVIĆA
GETALDIĆEVA 4, ZAGREB

ZAVRŠNI STRUČNI RAD

Program za praćenje zvijezda

MENTOR

Zlatko Nadarević dipl. ing.

UČENIK

Petar Babić 4.H

Zagreb, lipanj 2022.

Sadržaj

1	Uvod	1
2	Astrofotografija	2
2.1	Teleskop	2
2.1.1	Reflektorski teleskop	2
2.1.2	Refraktorski teleskop	3
2.2	Montaža teleskopa	3
2.2.1	Alt-Az montaža	3
2.2.2	Ekvatorijalna montaža	4
2.3	Nebeski ekvatorski koordinatni sustav	4
2.4	Kamera	5
2.4.1	Glavna kamera	5
2.4.2	Kamera za praćenje	5
2.5	Praćenje zvijezde	5
2.5.1	Odabir zvijezde	6
2.5.2	Kalibracija	6
3	Razvoj softvera	9
3.1	Pronalazak zvijezde	10
3.2	Kalibracija	13
3.2.1	Kalibracija kuta	14
3.2.2	Kalibracija jakosti	15
3.3	Praćenje	18
3.4	Kontrola kamere i montaže	20
4	Korištena oprema	23
5	Korištenje aplikacije	25
6	Rezultati	26
7	Zaključak	29
	Literatura	30
	Prilozi	31
	Datoteka control.py	31
	Datoteka camera.py	38
	Datoteka gui.py	40

1 Uvod

Astronomija je jedna od najstarijih prirodnih znanosti. Bavi se proučavanjem nebeskih objekata i pojava na noćnom nebu. Astronomiju možemo podijeliti na dvije grane: teorijska i promatračka. Teorijska astronomija usmjerena je na razvoj računalnih simulacija i modela za što točnije opisivanje i predviđanje astronomskih događaja, dok se promatračka astronomija bavi prikupljanjem podataka promatranjem maglica, planeta, zvijezda,..., te daljnjim analiziranjem tih podataka.

Da bismo promatrali nebeske objekte, nije dovoljno samo jednostavno usmjeriti teleskop u njih i snimati ih, već ih je potrebno i pratiti. Tu dolazimo do problema koji se ovim završnim radom pokušao riješiti.

2 Astrofotografija

Astrofotografija je grana fotografije koja se bavi fotografiranjem nebeskih objekata. Astrofotografija nam omogućuje snimanje oku nevidljivih tamnih objekata kao što su maglice i galaksije.

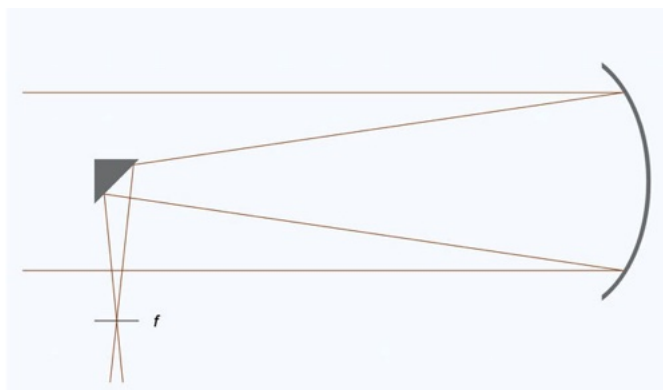
S obzirom na to da su nebeski objekti tamni potrebno je koristiti duge ekspozicije^[1] kako bi otkrili detalje koji se skrivaju u njima. U astrofotografiji uobičajene duljine ekspozicije kreću se od svega 30 s pa do 30 min. Zbog toga praćenje ima veliku ulogu u astrofotografiji, bez praćenja ne bi mogli snimati duge ekspozicije sa savršeno okruglim zvijezdama.

2.1 Teleskop

Teleskop je optički instrument koji se sastoji od leća, zakrivljenih zrcala ili kombinacije leća i zrcala. Većina teleskopa današnjice promatraju elektromagnetsko zračenje sve od X-zraka pa do radio valova i sve između. Dizajni samih teleskopa za promatranje elektromagnetskog zračenja ne razlikuju se toliko, koliko se razlikuju sami detektori. Dva glavna dizajna teleskopa su reflektori i refraktori.

2.1.1 Reflektorski teleskop

Reflektor je vrsta teleskopa koji se sastoji od dva zrcala, jednog zakrivljenog i jednog ravnog (Slika [1]). Postoje mnoge varijacije reflektorskog teleskopa (koriste dva zakrivljena zrcala, koriste kombinaciju leća i zrcala), kao što su: Newtonov, Gregorijanski, Schmidt-Cassegrain, Ritchey-Chrétien, ... Gotovo svi veći teleskopi korišteni u znanstvene svrhe su reflektori. Postoje mnogi razlozi za to, a najznačajniji su cijena i jednostavnost izrade. Neki od najpoznatijih reflektorskih teleskopa su: James Webb, Hubble.

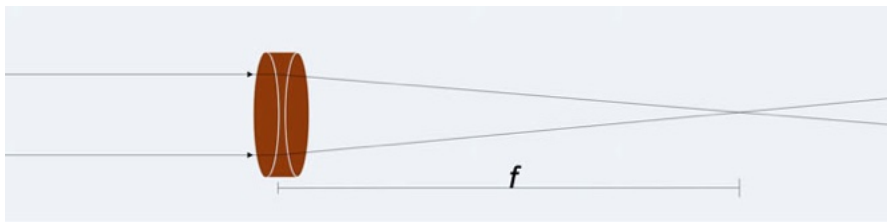


Slika 1: Reflektorski tip teleskopa [2]

¹Ekspozicija je vrijeme izlaganja fotografskog senzora svjetlosti.[1]

2.1.2 Refraktorski teleskop

Refraktor je optički teleskop koji se sastoji od dvije ili više leća (Slika 2). Koristi kombinaciju konvergentne i divergentne leće za smanjenje optičkih aberacija. Uobičajene veličine objektiva ovih teleskopa iznose od 50 mm pa sve do 200 mm. Refraktorski teleskopi većih objektiva postaju sve teži i skuplji u odnosu na reflektorske, te se zbog tog u zvjezdarnicama koriste reflektori ili teleskopi koji kombiniraju i leće i zakrivljena zrcala. Najveći refraktorski teleskop današnjice nalazi se u američkoj zvjezdarnici Yerkes i promjera je 102 cm [3].



Slika 2: Refraktorski teleskop [2]

2.2 Montaža teleskopa

Montaža je postolje na koje se postavlja teleskop i sva oprema spojena na teleskop, kako bi se teleskop mogao usmjeriti u nebo.

2.2.1 Alt-Az montaža

Alt-Az je jednostavna montaža s dvije rotacijske osi: horizontalnom (azimut) i vertikalnom (altitude) (Slika 3). Glavni nedostatak ove montaže je otežano praćenje nebeskih objekata. Praćenje nebeskih objekata otežano je zato što se oni rotiraju oko već poznate točke, a niti jedna os Alt-Az montaže nije paralelna s njom. Još jedan problem Alt-Az montaže je rotacija vidnog polja tijekom praćenja neba.



Slika 3: SkyWatcher AZ-5 montaža [4]

2.2.2 Ekvatorijalna montaža

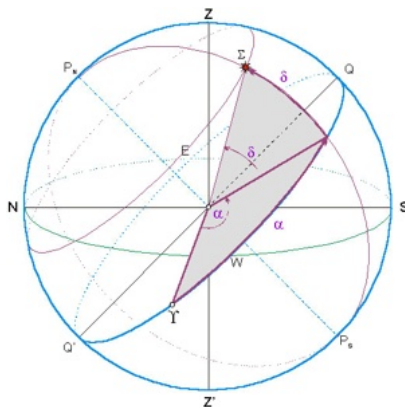
Ekvatorijalna montaža ima dvije osi: rektascenziju i deklinaciju (Slika 4). Ekvatorijalna montaža je slična kao i Alt-Az montaža, isto ima dvije rotacijske osi samo što je nagnuta pod određenim kutem i time azimutalna os postaje poravnata sa Zemljinom osi rotacije, tu os nazivamo rektascenzija. To je glavna prednost ekvatorijalne montaže, jer može kompenzirati rotaciju Zemlje i održavati isti nebeski objekt u vidnom polju teleskopa tijekom cijele noći.



Slika 4: SkyWatcher EQ6-r pro montaža [5]

2.3 Nebeski ekvatorski koordinatni sustav

Krug nebeskog ekvatora i ekvinocijski krug u nebeskom ekvatorskom sustavu čine primarni i sekundarni referentni krug. U ovom koordinatnom sustavu (Slika 5) položaj nebeskog tijela jednoznačno je određen rektascenzijom (α) i deklinacijom (δ). Rektascenzija je kut između ekvinocijskog kruga i satnog kruga nebeskog tijela (Σ) mjereno od proljetne točke (proljetnog ekvinocija) i po nebeskom ekvatoru u suprotnom smjeru dnevnog kretanja nebeskih tijela. Deklinacija je kut između kruga nebeskog ekvatora i smjera nebeskog tijela, mjereno po deklinacijskoj ili satnoj kružnici. [6].



Slika 5: Ekvatorski koordinatni sustav [6]

2.4 Kamera

Kako bismo mogli kvalitetno fotografirati nebo potrebne su dvije kamere: glavna kamera i kamera za praćenje. Glavna kamera koristi se za snimanje dugih ekspozicija, dok kamera za praćenje snima kraće ekspozicije i ispravlja sve nepravilnosti u praćenju neba.

2.4.1 Glavna kamera

Glavna kamera je kamera kojom se slikaju duge ekspozicije neba. Osjetljivije su i imaju senzore većih dimenzija i većih rezolucija od kamera za praćenje, te su još i hlađene kako bi se smanjio šum na slikama.

2.4.2 Kamera za praćenje

Kamera za praćenje je kamera kojom se kontinuirano slikaju kratke ekspozicije u rasponu od 0.5 s do 3 s i traži pomak zvijezde u odnosu na prethodnu sliku. Ako detektiramo promjenu položaja zvijezde montaži šaljem signal kako bi ispravili taj pomak.

2.5 Praćenje zvijezde

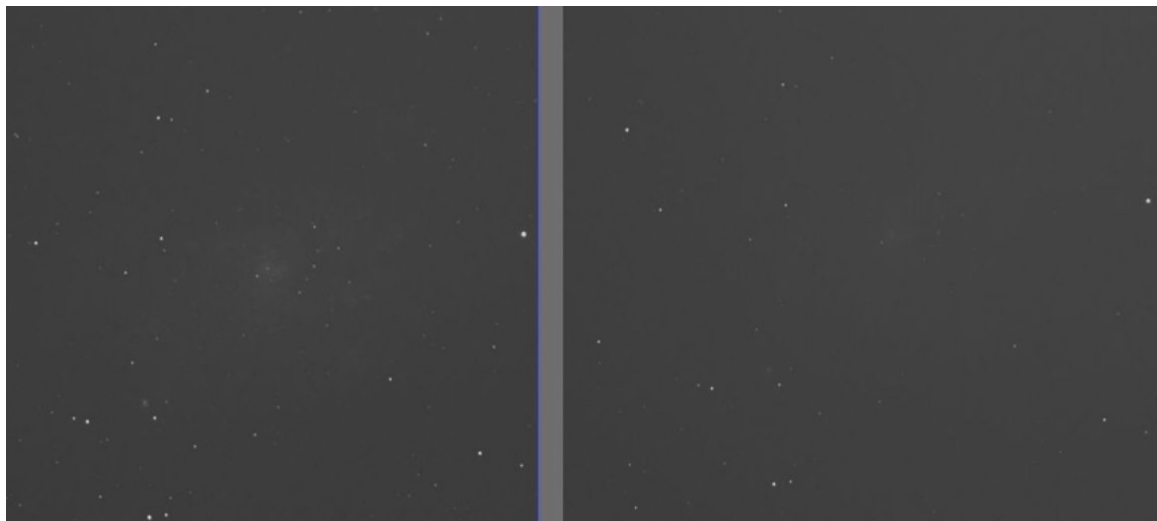
Praćenje zvijezde je sastavni dio astrofotografije. Zbog same rotacije Zemlje svi nebeski objekti kreću se istom brzinom oko točno definirane točke (u sjevernoj hemisferi to je sjeverni nebeski pol (blizu sjevernjače), u južnoj hemisferi to je južni nebeski pol). Prvi korak kako "suzbiti" rotaciju zemlje je korištenje ekvatorijalne montaže, montaže kojoj je jedna os poravnata sa Zemljinom rotacijom, a druga je okomita na prvobitnu os. Sljedeći korak je korištenje programa za praćenje. Glavna svrha programa za praćenje je ispravak svih mogućih pogrešaka koje pridonose nepravilnosti u mehanizaciji ekvatorijalne montaže i netočnom poravnanju rektascenzijske osi s nebeskim polom. Praćenje zvijezde je proces kojim se pomoću kamere za praćenje snima ekspozicije u kratkim intervalima uspoređujući položaj zvijezde na svakoj slici s početnom ne bi li se pronašao pomak zvijezde i korigirao taj pomak kako bi zvijezda ostala savršeno okrugla na dugoj ekspoziciji glavne kamere (Slika 6).



Slika 6: Usporedba duge ekspozicije bez praćenja (lijevo) i s praćenjem (desno) [7]

2.5.1 Odabir zvijezde

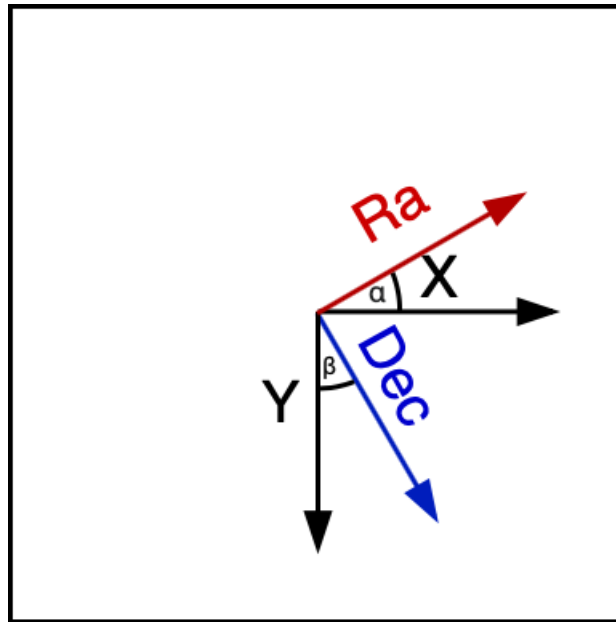
Kod odabira zvijezde za praćenje važno je paziti na to da je odabrana zvijezda prikladne veličine i da se ne nalazi blizu ruba slike. Ako je prevelika program će teže opaziti male pomake zvijezde, a ako je premalena postoji mogućnost da će zvijezda nestati iza "tankih" oblaka (Slika 7) ili ju program neće moći detektirati. Ako se zvijezda nalazi blizu ruba slike, postoji mogućnost da malo jači nalet vjetra pomakne montažu i prividno „otpuše” zvijezdu van vidnog polja kamere za praćenje.



Slika 7: Usporedba fotografije s (lijevo) i bez visokih oblaka (desno) [8]

2.5.2 Kalibracija

S obzirom na neznanje orijentacije kamere u odnosu na osi montaže, tj. nebeskog ekvatorskog sustava, potrebno je odrediti dva kuta: α (kut između rektascenzije (Ra) i x osi) i β (kut između deklinacije (Dec) i y osi) (Slika 8). Ove kuteve potrebno je odrediti zbog mogućih netočnosti u određenim koordinatama zvijezde na slici. Naknadno, uspoređujući ta dva kuta, možemo ustanoviti točniji kut ako je devijacija između rezultata mala (izračuna se srednja vrijednost). No, ako je razlika između kuteva velika, ponovno se pokreće postupak kalibracije.



Slika 8: Kut između x i y osi i rektascenzije i deklinacije

Kut α pronalazi se pomicanjem montaže u smjeru rektascenzije i fotografiranjem slike. Odrede se koordinate zvijezde i kut se tada računa korištenjem funkcije $\arctan2$ ² dostupne u svim većim programskim jezicima:

$$\alpha = \arctan2(y_1 - 16, x_1 - 16) \cdot \frac{180^\circ}{\pi}$$

Kut β pronalazi se na isti način, ali je potrebno zamijeniti x i y osi u funkciji i pomnožiti ih s -1 jer se na slikama y-os gleda vertikalno prema dolje:

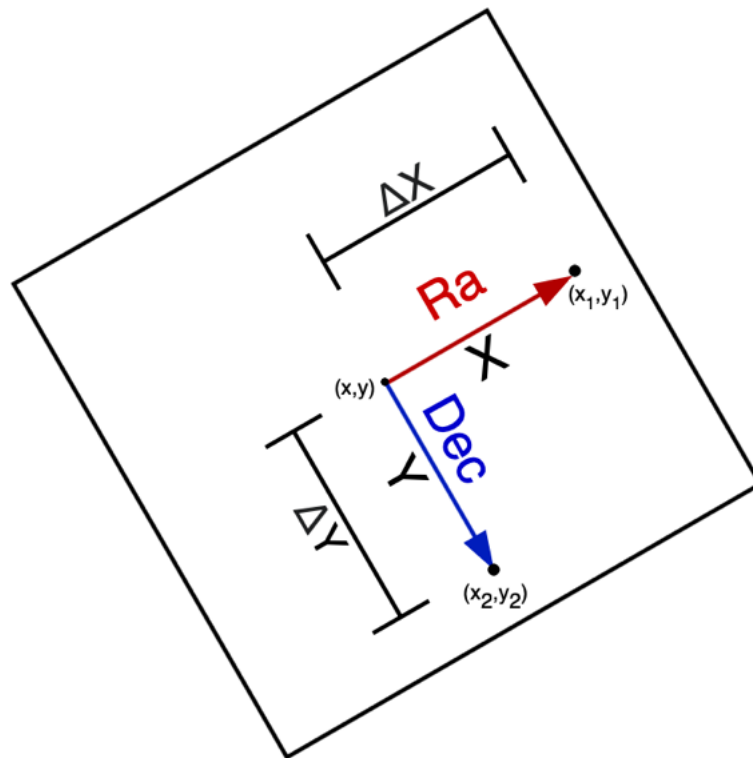
$$\beta = \arctan2(-(x_2 - 16), -(y_2 - 16)) \cdot \frac{180^\circ}{\pi}$$

Ako je devijacija između kuteva mala, konačna vrijednost kuta (γ) računa se prema formuli:

$$\gamma = \frac{\alpha + \beta}{2}$$

Zadnji korak kalibracije je određivanje broja piksela za koje se zvijezda pomiče kada se montaža pomiče po svojim osima (Ra i Dec) (Slika 9). S obzirom na to da je u prethodnom koraku određen kut pod kojim je kamera rotirana u odnosu na smjer pomicanja montaže, potrebno je rotirati sliku za taj kut i promatrati samo x ili y os, ovisno koja se os pomiče.

²Funkcija $\arctan2$ definira se kao kut radijanima između pozitivne x osi i zraka do točke (x,y)[9]



Slika 9: Udaljenost

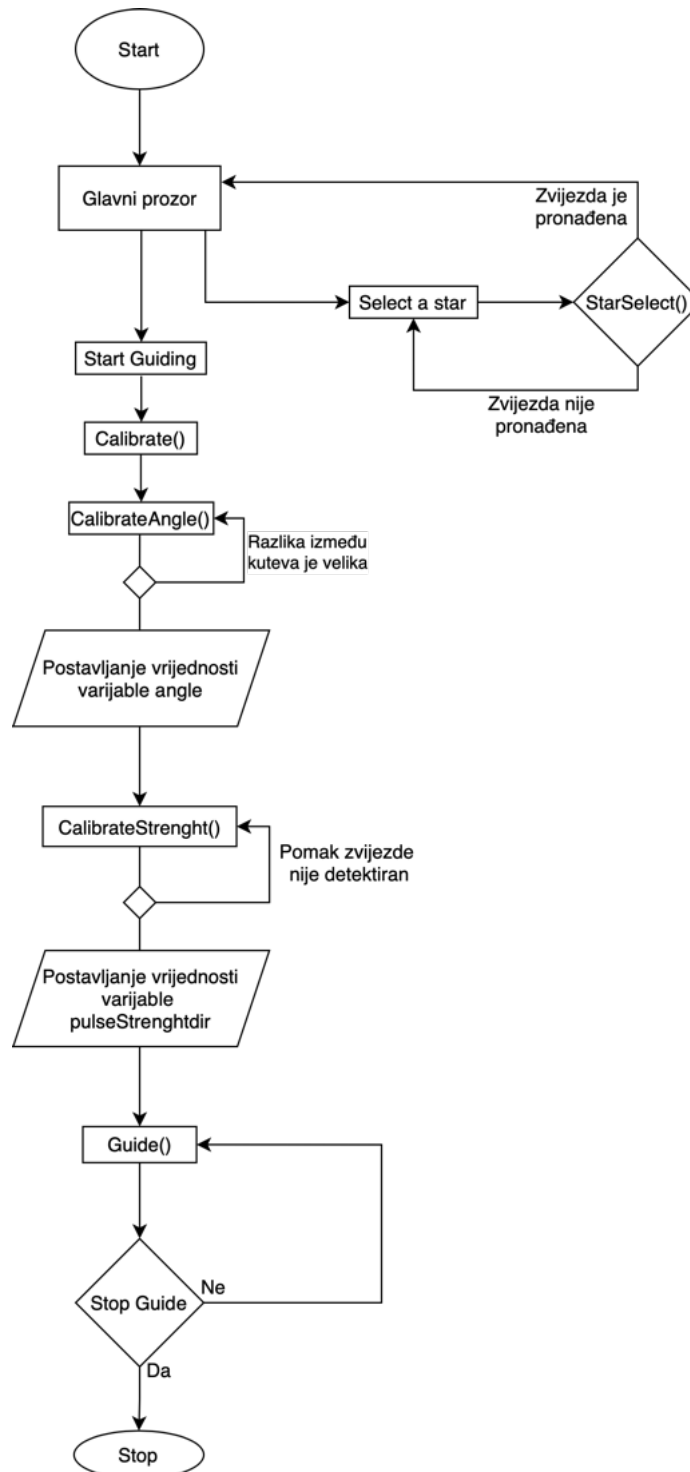
Kada se pomiče os deklinacije, promatra se razlika piksela po y osi:

$$\Delta y = |y_2 - y|$$

U slučaju kada se pomiče os rektascenzije, promatra se razlika piksela po x osi:

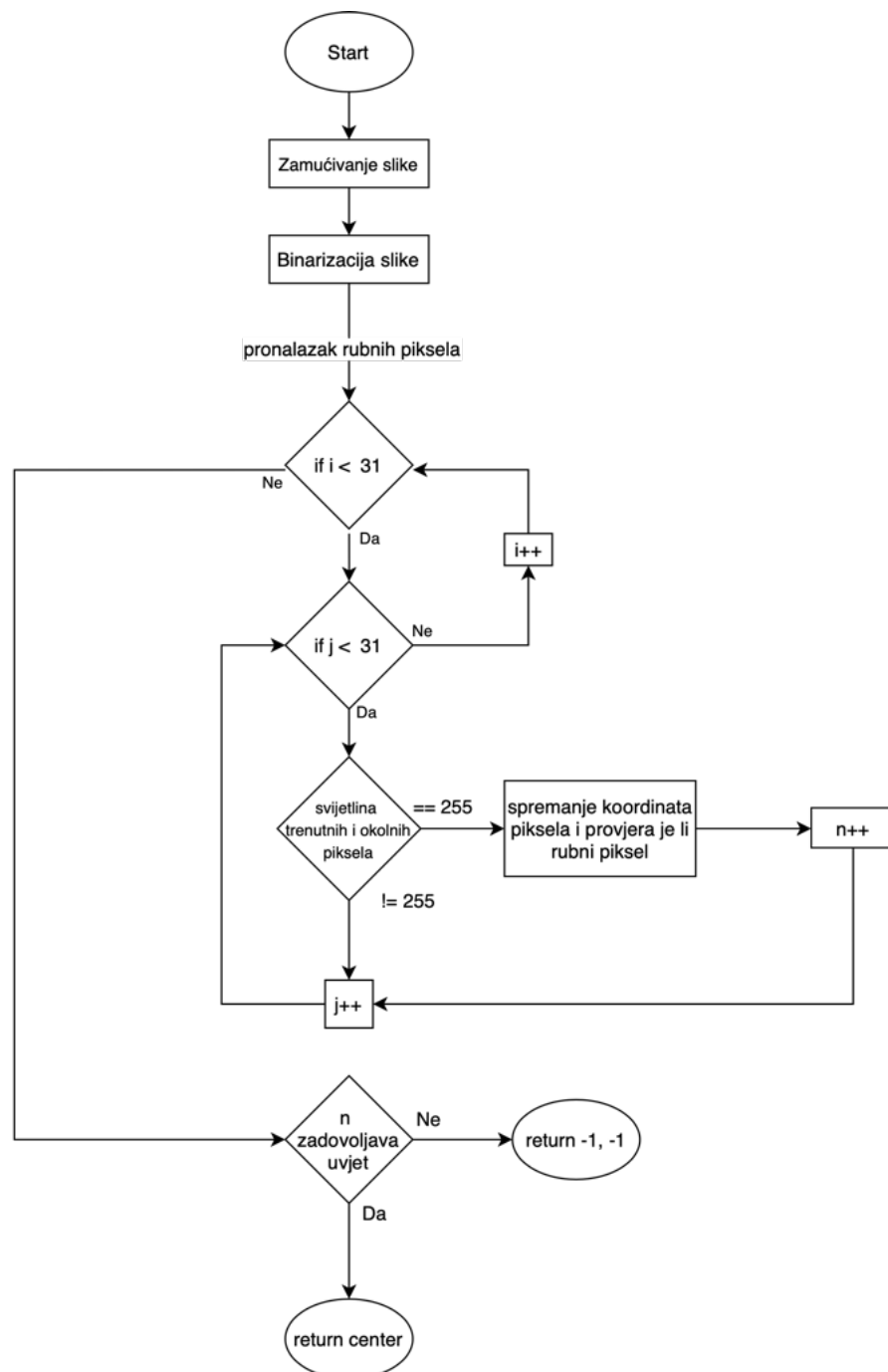
$$\Delta x = |x_1 - x|$$

3 Razvoj softvera



Slika 10: Blok dijagram glavnog programa

3.1 Pronalazak zvijezde



Slika 11: Blok dijagram funkcije StarCenter()

Pronalazak zvijezde odvija se u dva koraka: prvi je grubi pronalazak (korisnikov odabir zvijezde koja će se pratiti). Nakon toga slijedi pronalazak točnih x, y koordinata zvijezde na slici.

Korisnik zvijezdu na slici odabire pomoću GUI-a. Pritiskom na tipku "Star Select" korisnik će odabrati dio slike (32x32 piksela) na kojoj se nalazi potencijalna zvijezda te će

se te koordinate proslijediti datoteci control.py i metoda StarSelect će pronaći precizne koordinate odabrane zvijezde, naravno ako zvijezda postoji u tom dijelu slike.

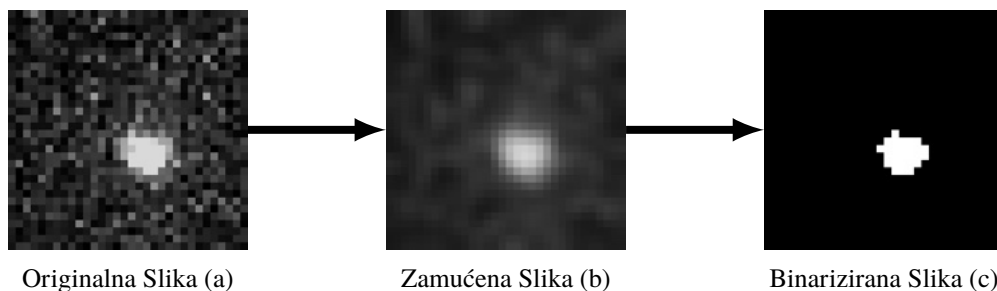
Metoda StarSelect prima dva argumenta pozicije (x, y) na slici i vraća True ili False ako je/nije pronađena zvijezda na slici, te postavlja vrijednosti varijabli posX i posY kako bi kasnije program znao gdje se zvijezda nalazi.

```

1 def StarSelect(self, posX, posY):
2     image = cv.imread("./image_mono.tiff")
3
4     roi = image[posY - 16:posY + 16, posX - 16:posX + 16]
5
6     y1, x1 = self.StarCenter(roi)
7
8     posX += x1 - 16
9     posY += y1 - 16
10
11     roi = image[posY - 16:posY + 16, posX - 16:posX + 16]
12
13     if(y1 != -1 and x1 != -1):
14         self.starX = posX
15         self.starY = posY
16
17         print(self.starX)
18         print(self.starY)
19         return True
20     else:
21         return False

```

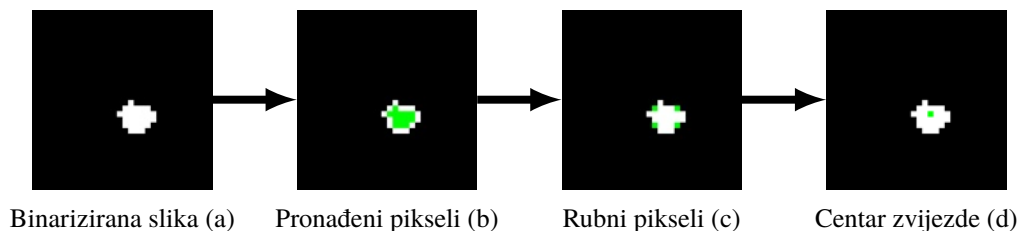
Metoda StarSelect pozvat će metodu StarCenter kojoj je glavna funkcija pronalazak koordinata zvijezde na slici. Sliku je prvo potrebno zamutiti pa binarizirati kako bi se lakše pronašla zvijezda. (Slika [12](#)). Binarizacija je proces koji svaki piksel postavlja na dvije različite vrijednosti, ovisno o vrijednosti praga. Ako je vrijednost piksela manja od praga, postavlja se na 0, inače se postavlja na maksimalnu vrijednost (255). [\[10\]](#)



Slika 12: Obrada slike

Nakon toga, pomoću petlje, prolazi se kroz sve vrijednosti piksela i tražimo piksele

koji imaju vrijednost 255. Slijedi provjera nalazi li se taj piksel na rubu zvijezde kako bi kasnije olakšali računanje pronalaska centra zvijezde (Slika 13.). Osim toga, unutar same petlje, broji se broj pronađenih piksela za slučaj da na slici ili ne postoji zvijezda ili je zvijezda prevelika/premala.



Slika 13: Pronalazak centra zvijezde

```

1 for i in range(1, x - 1):
2     for j in range(1, y - 1):
3         # Looks if current pixel and pixels around it are brighter
           than 255
4         if(binarized.item(j + 1, i, 1) >= 255 and binarized.item(j - 1,
           i, 1) >= 255 and binarized.item(j, i + 1, 1) >= 255 and
           binarized.item(j, i - 1, 1) >= 255 and
5         binarized.item(j, i, 1) >= 255):
6             if(j > maxY):
7                 maxY = j
8             if(i > maxX):
9                 maxX = i
10            if(j < minY):
11                minY = j
12            if(i < minX):
13                minX = i
14
15            n += 1

```

Zadnji korak u pronalasku zvijezde je jednostavan račun centra zvijezde. Glavni uvjet za započinjanje računanja centra je provjera da:

- je broj pronađenih piksela veći od tri
- je broj pronađenih piksela je manji od 70% ukupnog broja piksela
- na slici postoje crni pikseli.

S obzirom na to da su rubne koordinate već poznate, potrebno ih je samo zbrojiti i podijeliti s dva:

$$x_{centar} = \frac{x_{lijevo} + x_{desno}}{2}$$

$$y_{centar} = \frac{y_{gore} + y_{dolje}}{2}$$

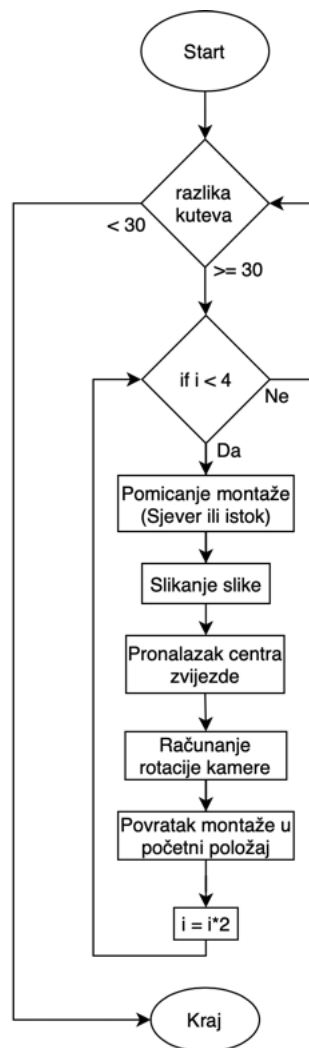
```
1 if(n >= 3 and n < x * y - x * y * 0.3 and np.min(binarized) == 0):
2     # Izracun koordinata centra zvijezde
3     center = int((minY + maxY) / 2), int((minX + maxX) / 2)
4
5     if(debug):
6         print(center)
7
8     return center
9
10 else:
11     print("No stars found")
12     return -1, -1
```

3.2 Kalibracija

S obzirom na to da kamera neće svaki put biti isto orijentirana u odnosu na gibanje neba i montaže te na netočno balansiranje montaže, potrebno je izvršiti kalibraciju smjera i jakosti svaki put kada se montaža pomakne na udaljeni dio neba.

Prvo je potrebno napraviti kalibraciju smjera/kuta kako bi za vrijeme kalibracije jakosti jednog impulsa znali u kojem se točno smjeru pomiče montaža.

3.2.1 Kalibracija kuta

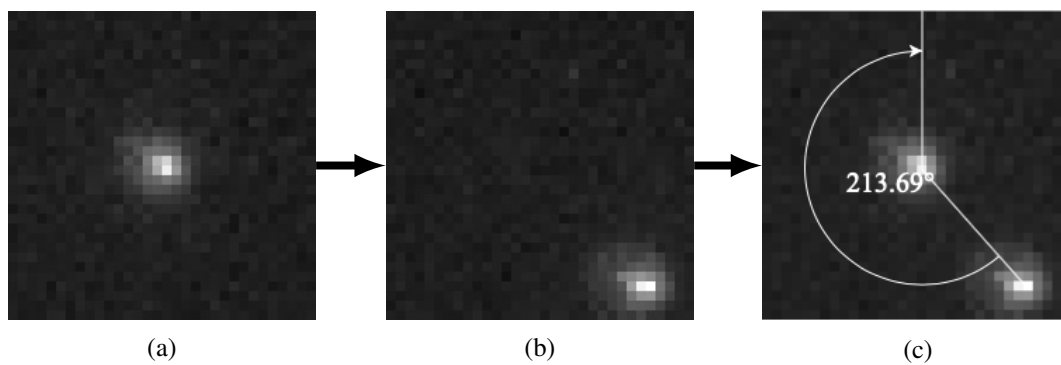


Slika 14: Blok dijagram funkcije CalibrateAngle()

Kalibraciju smjera izvršava metoda CalibrateAngle unutar klase Guider. Kalibracija smjera provodi se tako da se fotografira početna slika (15.a), pomakne montaža u već predefiniranom smjeru, ponovno fotografira slika (15.b) i pronade kut pod kojim se zvijezda pomaknula (Slika 15.c).

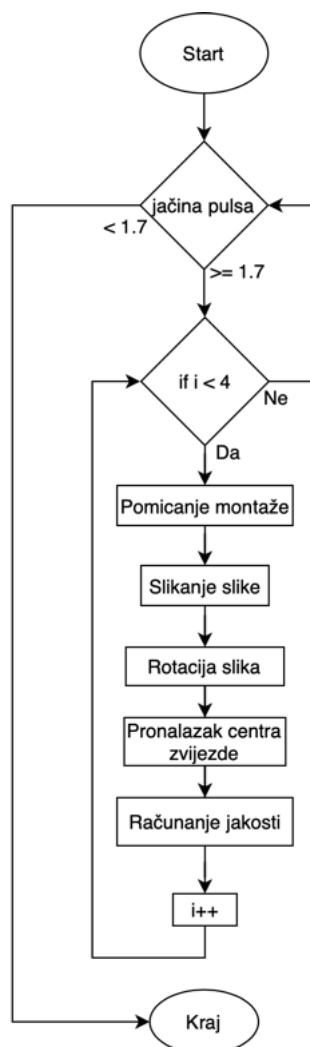
```

1 angle1 = 360 - np.abs(np.rad2deg(np.arctan2((x2 - 16) * -1, (y2 - 16)
    * -1)))
2 angle2 = 360 - np.abs(np.rad2deg(np.arctan2(y2 - 16, x2 - 16)))
  
```

Slika 15: Pronalazak kuta

3.2.2 Kalibracija jakosti



Slika 16: Blok dijagram funkcije CalibrateStrenght()

Jakost jednog guiding pulsa računa se na jako sličan način kao i kalibracija kuta. Fotografira se početna sliku (Slika 17a), nakon toga montaži se šalje guiding puls određenog

trajanja u određenom smjeru i fotografira se nova slika (Slika 17.b). S obzirom da je već poznato pod kojim kutem bi se zvijezda trebala pomaknuti, za taj iznos rotira se sliku (Slika 17.c) pomoću funkcije `RotateImage()`.

```

1 camera.TimedGuide(self.dir[i], 5)
2
3 image = cv.imread(camera.capture(1, 75))
4 xOld = self.starX
5 yOld = self.starY
6
7 roi = image[self.starY - 16:self.starY + 16, self.starX - 16:self.
      starX + 16]
8 self.starX += self.StarCenter(roi)[1] - 16
9 self.starY += self.StarCenter(roi)[0] - 16
10
11 image = self.RotateImage(image, self.angle, xOld, yOld)

```

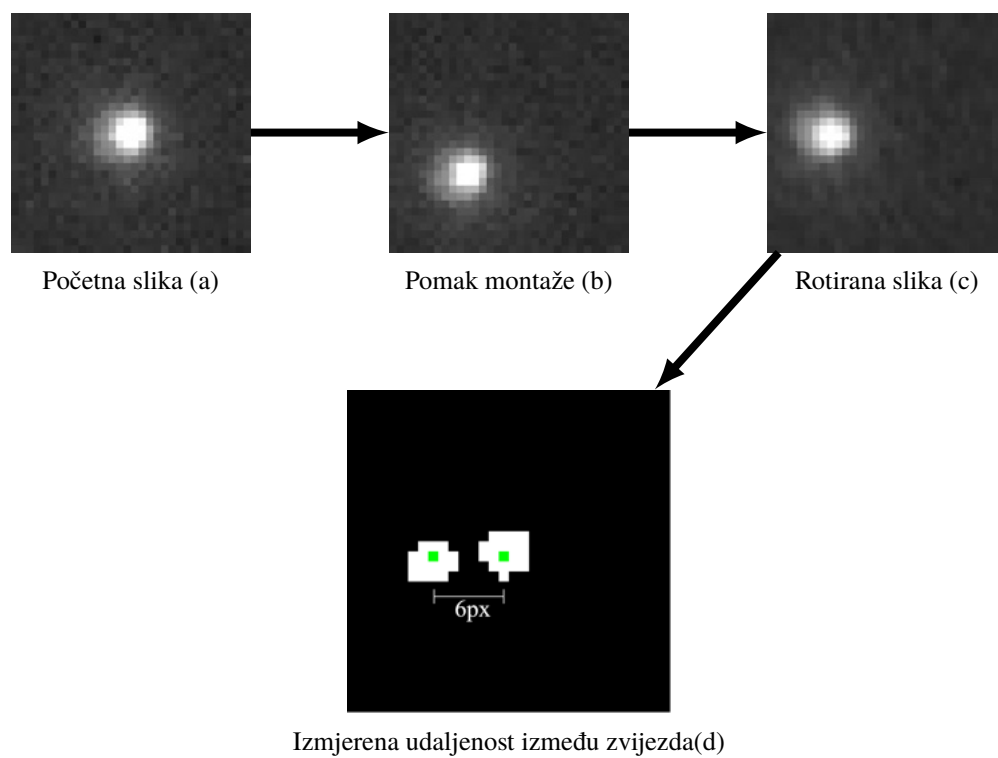
Nakon rotacije moguće je jednostavno izmjeriti udaljenost od prvobitnog centra zvijezde do trenutnog centra zvijezde, gledajući samo x ili y osi ovisno o smjeru pomicanja montaže (Slika 17.d). Kako bi se izračunala jakost jednog guiding pulsa, potrebno je podijeliti trajanje tog pulsa s brojem piksela za koliko se pomaknula zvijezda:

$$j = \frac{t[s]}{n[px]}$$

```

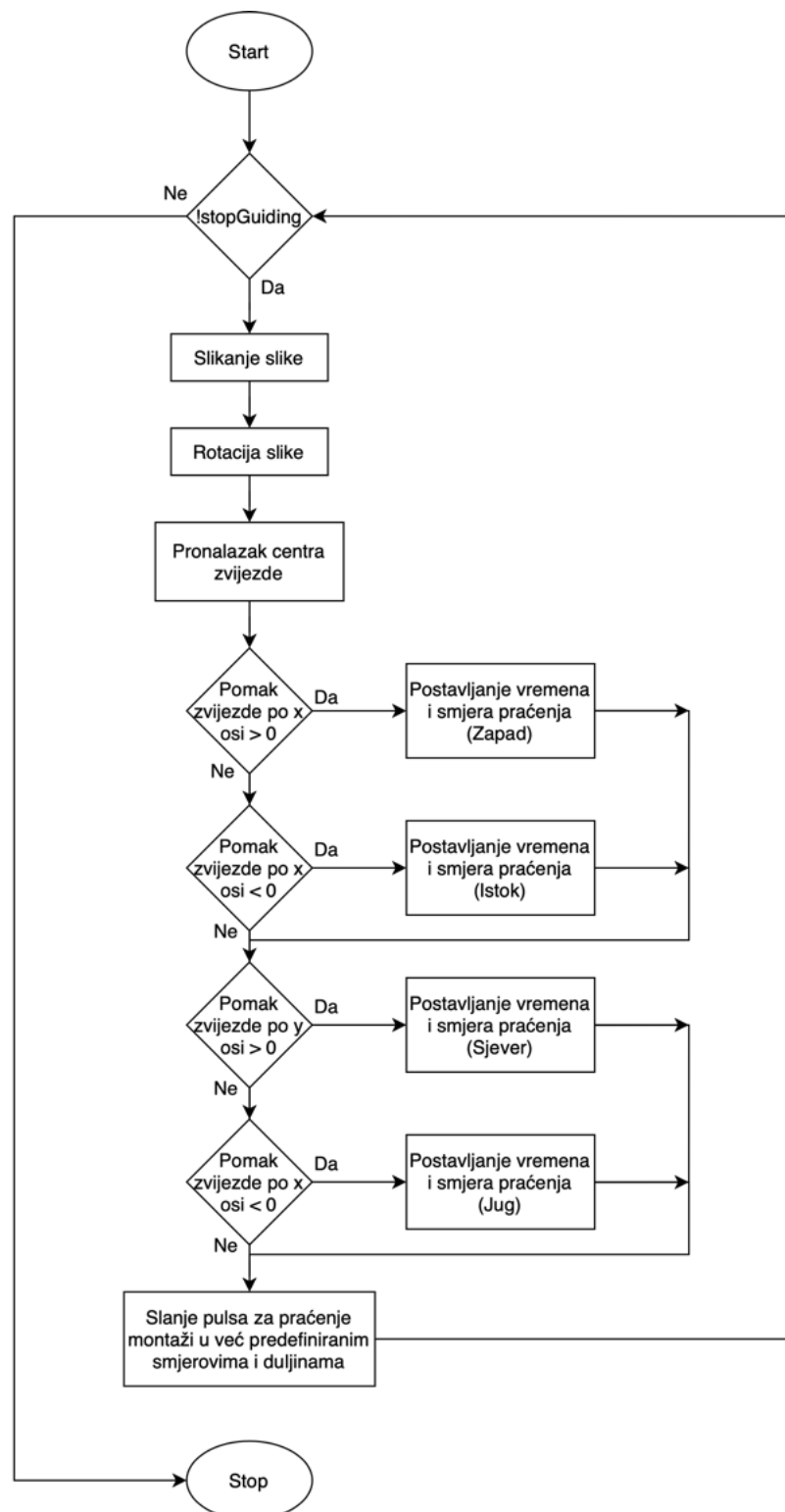
1 roi = image[yOld - 16:yOld + 16, xOld - 16:xOld + 16]
2
3 dirY, dirX = self.StarCenter(roi)
4
5 if(i <= 1):
6     if(dirY != 16):
7         self.pulseStrenghtdir[i] = np.abs(5 / (dirY - 16))
8 else:
9     if(dirX != 16):
10        self.pulseStrenghtdir[i] = np.abs(5 / (dirX - 16))

```



Slika 17: Udaljenost zvijezde

3.3 Praćenje



Slika 18: Blok dijagram funkcije Guide()

Praćenje je proces koji se kontinuirano odvija sve dok ne bude prekinut ili od strane korisnika ili zbog gubitka zvijezde. Tijekom praćenja zvijezde program konstantno fotografira

slike, kalibrira ih te pronalazi koordinate centra zvijezde i uspoređuje te koordinate s početnim koordinatama ne bi li pronašao pomak zvijezde.

```

1 while not self.stopGuiding:
2     image = cv.imread("./image_mono.tiff")
3
4     image = self.Calibrate(image)
5     roi = image[self.starY - 16:self.starY + 16, self.starX - 16:self.
        starX + 16]
6
7     self.dirY, self.dirX = self.StarCenter(roi)

```

Ako postoji pomak, program šalje guiding puls montaži kako bi vratio zvijezdu u centar slike. Ovisno o smjeru pomaka zvijezde u varijable maxDir i minDir postavljamo smjer u kojem je potrebno pomaknuti montažu kako bi vratili zvijezdu u centar slike, dok se u varijable maxTime i minTime postavlja trajanje pomicanja montaže, to vrijeme računamo po formuli:

$$t = j \cdot n$$

```

1 if(self.dirY != -1 and self.dirX != -1):
2     self.dirX -= 16
3     self.dirY -= 16
4
5     if(self.dirX > 0):
6         maxTime = self.pulseStrenghtdir[3] * np.abs(self.dirX)
7
8         maxDir = "W"
9     elif(self.dirX < 0):
10        maxTime = self.pulseStrenghtdir[2] * np.abs(self.dirX)
11
12        maxDir = "E"
13    else:
14        maxTime = 0
15
16    if(self.dirY > 0):
17        minTime = self.pulseStrenghtdir[0] * np.abs(self.dirY)
18
19        minDir = "N"
20    elif(self.dirY < 0):
21        minTime = self.pulseStrenghtdir[1] * np.abs(self.dirY)
22
23        minDir = "S"
24    else:
25        minTime = 0

```

Gledamo trajanje impulsa, te ako je vrijednost minTime veća od maxTime, zamjene se

vrijednosti. I na kraju se montaža pomiče u smjeru i trajanju koje je definirano u prethodno spomenutim varijablama.

```

1 if(minTime > maxTime):
2     temp = maxTime
3     maxTime = minTime
4     minTime = temp
5
6     temp = maxDir
7     maxDir = minDir
8     minDir = temp
9
10    starTime = time.time()
11    camera.StartGuide(maxDir)
12    camera.StartGuide(minDir)
13
14    if(maxTime > 2): maxTime = 2
15    if(minTime > 2): minTime = 2
16
17    if(maxTime != 0):
18        while(time.time() < starTime + maxTime):
19            if(time.time() > starTime + minTime):
20                camera.StopGuide(minDir)
21                camera.StopGuide(maxDir)

```

3.4 Kontrola kamere i montaže

Kameru ZWO ASI 120mm-mini (preko koje se šalju guiding pulsevi montaži HEQ5) kontroliramo pomoću datoteke `camera.py`. Moguće je kontrolirati duljinu ekspozicije i gaina. Gain je analogno pojačanje signala koje je kamera uhvatila tijekom snimanja ekspozicije. Što je veći broj gaina slika je svjetlija i obratno, što je manji gain to je slika tamnija. Kod postavljanja gaina treba biti oprezan kako se ne bi postavila vrlo niska ili vrlo visoka vrijednost. Rezultat toga je da je fotografirana slika potpuno crna ili potpuno bijela.

Ekspozicija i gain kontroliraju se pomoću funkcije `capture` koja prima dva argumenta `exposure` - duljina ekspozicije i `gain` - vrijednost osjetljivosti, te vraća putanju do datoteke spremljene slike.

```

1 def capture(expouser, gain):
2     while(True):
3         camera.set_control_value(asi.ASI_GAIN, gain)
4         camera.set_control_value(asi.ASI_EXPOSURE, int(expouser * 1000
5             000)) # microseconds
6
7         filename = 'image_mono.tiff'
8         camera.set_image_type(asi.ASI_IMG_RAW8)

```

```
8         if(camera.capture(filename=filename)):  
9             break  
10  
11     return "./image_mono.tiff"
```

Guiding pulseve pulsevi šalju se putem kamere koja te pulseve prosljeđuje montaži preko ST-4 kabela. Guiding pulseve moguće je poslati na dva načina. Ako se želi poslati jedan po jedan impuls koristi se funkcija TimeGuide. U slučaju kada se želi montažu pomicati paralelno u dva smjera (npr. sjever i istok), poziva se funkcija StartGuide() za oba smjera te se nakon određenog vremena praćenje zaustavlja pozivom funkcije StopGuide().

```
1 def TimeGuide(direction, duration):  
2     if direction == "N":  
3         camera.pulse_guide_on(asi.ASI_GUIDE_NORTH)  
4     elif direction == "S":  
5         camera.pulse_guide_on(asi.ASI_GUIDE_SOUTH)  
6     elif direction == "E":  
7         camera.pulse_guide_on(asi.ASI_GUIDE_EAST)  
8     elif direction == "W":  
9         camera.pulse_guide_on(asi.ASI_GUIDE_WEST)  
10  
11     print("Moving: " + direction)  
12  
13     time.sleep(duration)  
14  
15     camera.pulse_guide_off(asi.ASI_GUIDE_NORTH)  
16     camera.pulse_guide_off(asi.ASI_GUIDE_SOUTH)  
17     camera.pulse_guide_off(asi.ASI_GUIDE_EAST)  
18     camera.pulse_guide_off(asi.ASI_GUIDE_WEST)  
19  
20     time.sleep(duration)
```

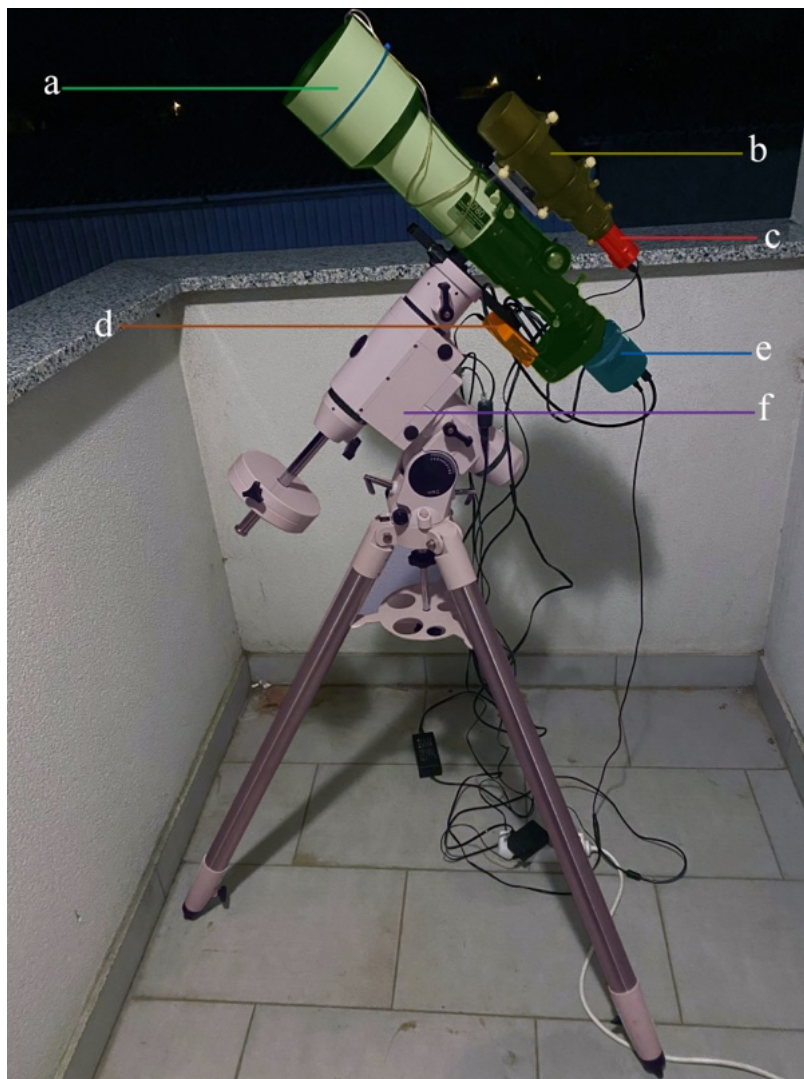
```
1 def StartGuide(direction):  
2     if direction == "N":  
3         camera.pulse_guide_on(asi.ASI_GUIDE_NORTH)  
4     elif direction == "S":  
5         camera.pulse_guide_on(asi.ASI_GUIDE_SOUTH)  
6     elif direction == "E":  
7         camera.pulse_guide_on(asi.ASI_GUIDE_EAST)  
8     elif direction == "W":  
9         camera.pulse_guide_on(asi.ASI_GUIDE_WEST)  
10  
11 def StopGuide(direction):  
12     if direction == "N":  
13         camera.pulse_guide_off(asi.ASI_GUIDE_NORTH)  
14     elif direction == "S":  
15         camera.pulse_guide_off(asi.ASI_GUIDE_SOUTH)
```

```
16     elif direction == "E":  
17         camera.pulse_guide_off(asi.ASI_GUIDE_EAST)  
18     elif direction == "W":  
19         camera.pulse_guide_off(asi.ASI_GUIDE_WEST)
```


4 Korištena oprema

U ovom radu korištena je sljedeća oprema:

- Teleskop - Explore Scientific ED80 (Slika 19.a)
 - fokalna Daljina: 480 mm
 - promjer objektiva: 80 mm
- Teleskop za praćenje (Slika 19.b)
 - fokalna Daljina: 180 mm
 - promjer objektiva: 50 mm
- Kamera za praćenje - ZWO ASI 120MM-mini (Slika 19.c)
 - veličina senzora: 1/3"
 - rezolucija: 1280 x 960 piksela
 - tip senzora: monokromatski
- Raspberry Pi 4b (Slika 19.d)
- Glavna kamera - QHY163M (Slika 19.e)
 - veličina senzora: 4/3"
 - rezolucija: 4656 x 3522 piksela
 - tip senzora: monokromatski
- Montaža - Sky-Watcher HEQ5 PRO (Slika 19.f)
- Filter - Baader H-alpha Ultra-Narrowband-Filters (3.5nm) - CMOS-optimized
- Filter - Baader OIII Narrowband-Filters (6.5nm) - CMOS-optimized

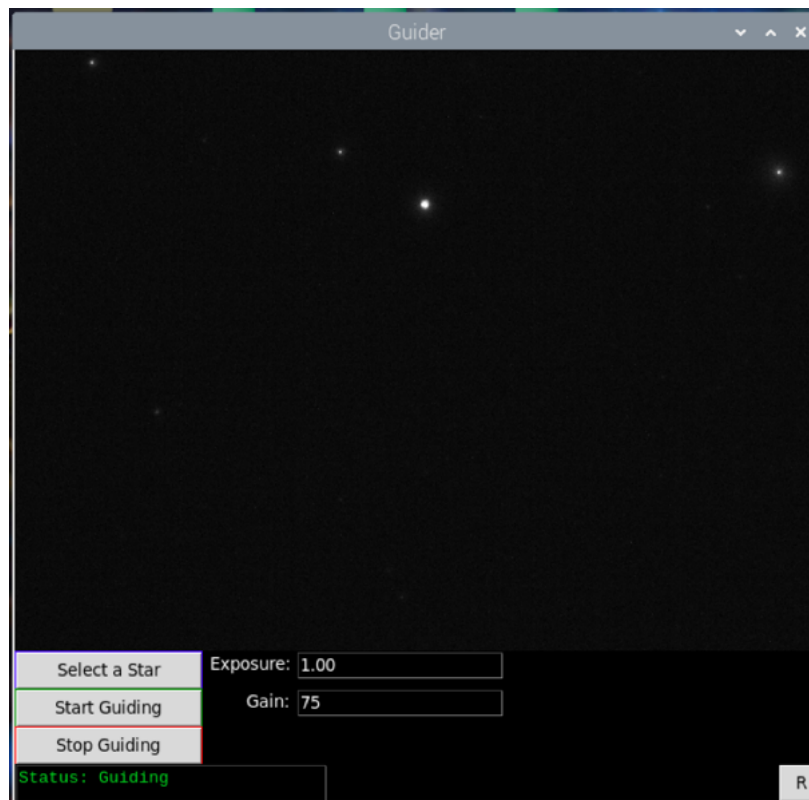


Slika 19: Korištena oprema

Glavna komponenta opreme je Sky-Watcher HEQ5 PRO montaža. Bez navedene montaže fotografiranje bi bilo nemoguće. Montaža radi na principu da joj je os rektascenzije paralelna s osi rotacije Zemlje i to joj omogućuje praćenje noćnog neba. Kako bi se postiglo točno praćenje noćnog neba prvo je potrebno precizno "usjeveriti"³ montažu. Proces usjeveravanja nikada nije potpuno točan i potrebno je koristiti i os deklinaciju kako bi se ispravile eventualne pogreške u praćenju, odnosno male nepreciznosti u mehanizmu montaže i usjeveravanju. Maksimalna nosivost montaže iznosi 11 kg. Kamera za praćenje spojena je na montažu putem ST-4 guiding porta. Opremom se upravlja pomoću Raspberry Pi 4b.

³Poravnavanje osi rektascenzija sa Zemljinom osi rotacije

5 Korištenje aplikacije



Slika 20: GUI programa

Pritiskom na gumb Select a Star korisnik odabire zvijezde koju želi da program prati tokom sesije.

Pritiskom na gumb Start Guiding započinje se praćenje zvijezde (prvo je kalibracija, te nakon toga praćenje).

Pritiskom na gumb Stop Guiding zaustavlja se praćenje.

U tekstualni okvir desno od Exposure unosi se duljina ekspozicije kojom će kamera snimati.

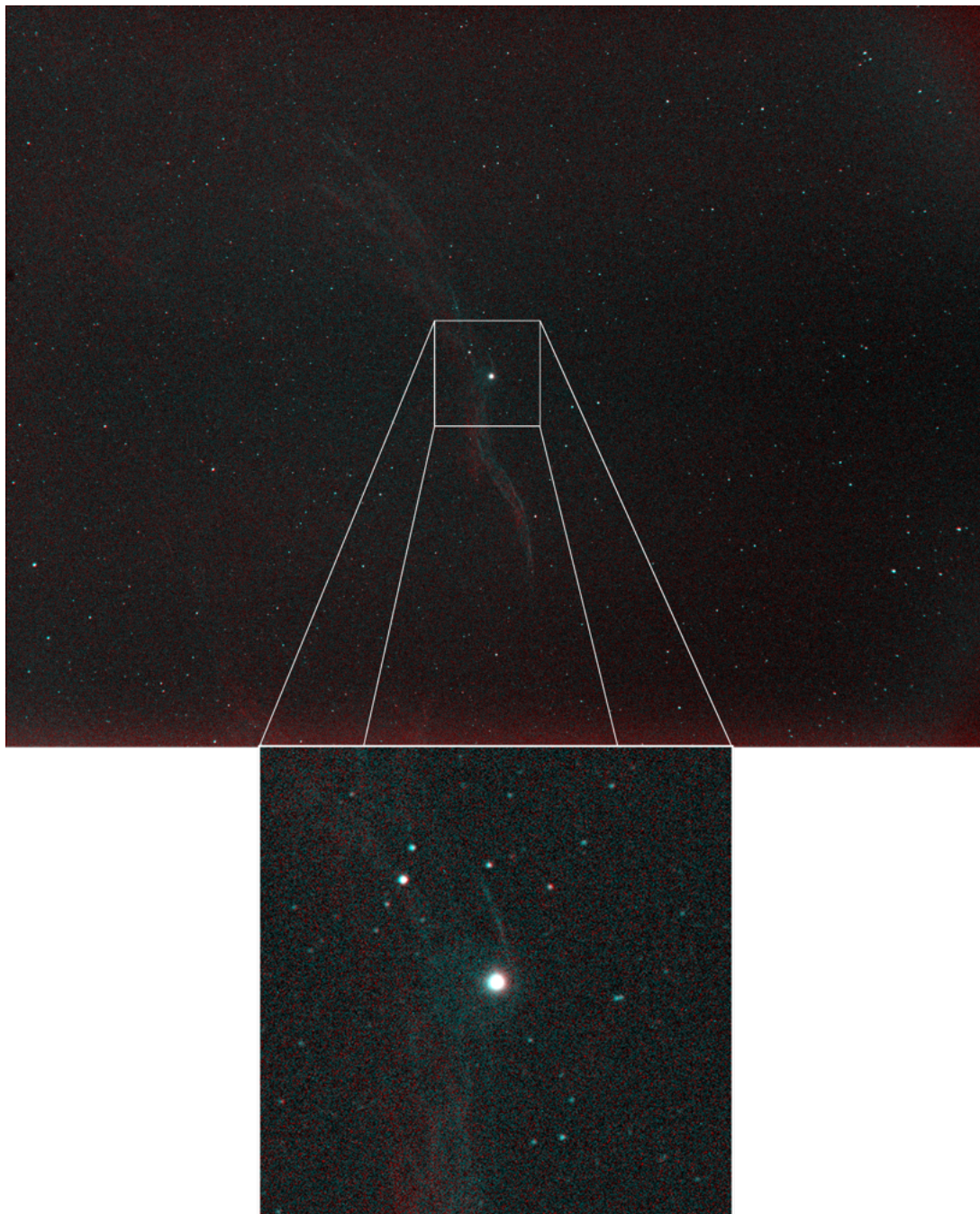
U tekstualni okvir desno od Gain unosi se vrijednost gain-a.

Tekstualni okvir ispod tipke Stop Guiding prikazuje trenutno stanje programa ("Guiding", "Guiding stopped", "Selecting a star", "Found a star", "No stars found", "Calibrating")

Pritiskom na gumb R ponovno pokrećemo aplikaciju

6 Rezultati

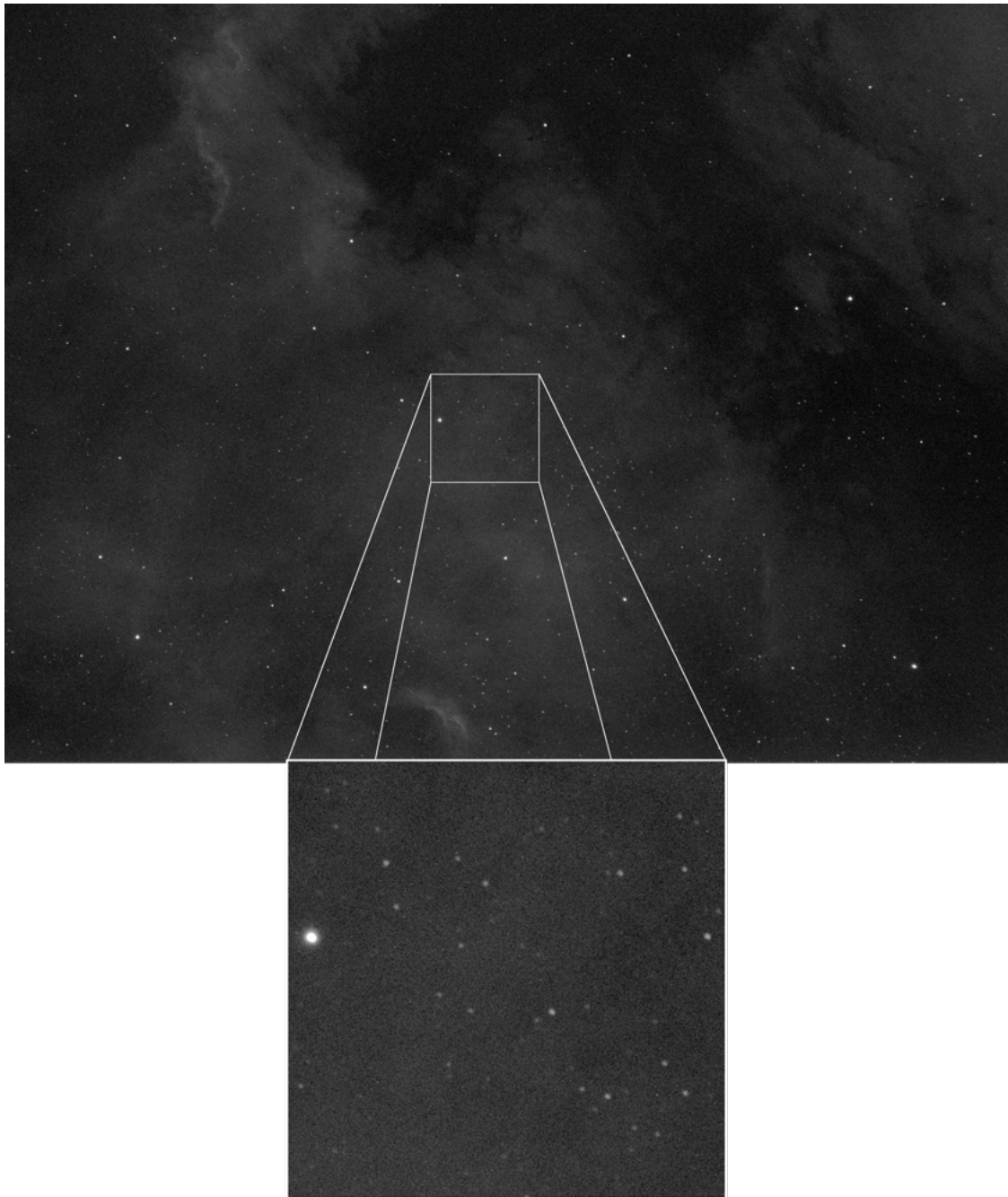
Sve prikazane fotografije, fotografirane su korištenjem prethodno navedene opreme (poglavlje 4). Slika 21 je HOO⁴ kombinacija. Slika fotografirana kroz Ha filter duljine ekspozicije 300 s, gain-a 200 i senzor ohlađen na 0 °C, a ekspozicija fotografirana kroz OIII filter je imala potpuno iste vrijednosti osim što je senzor bio ohlađen na –20 °C.



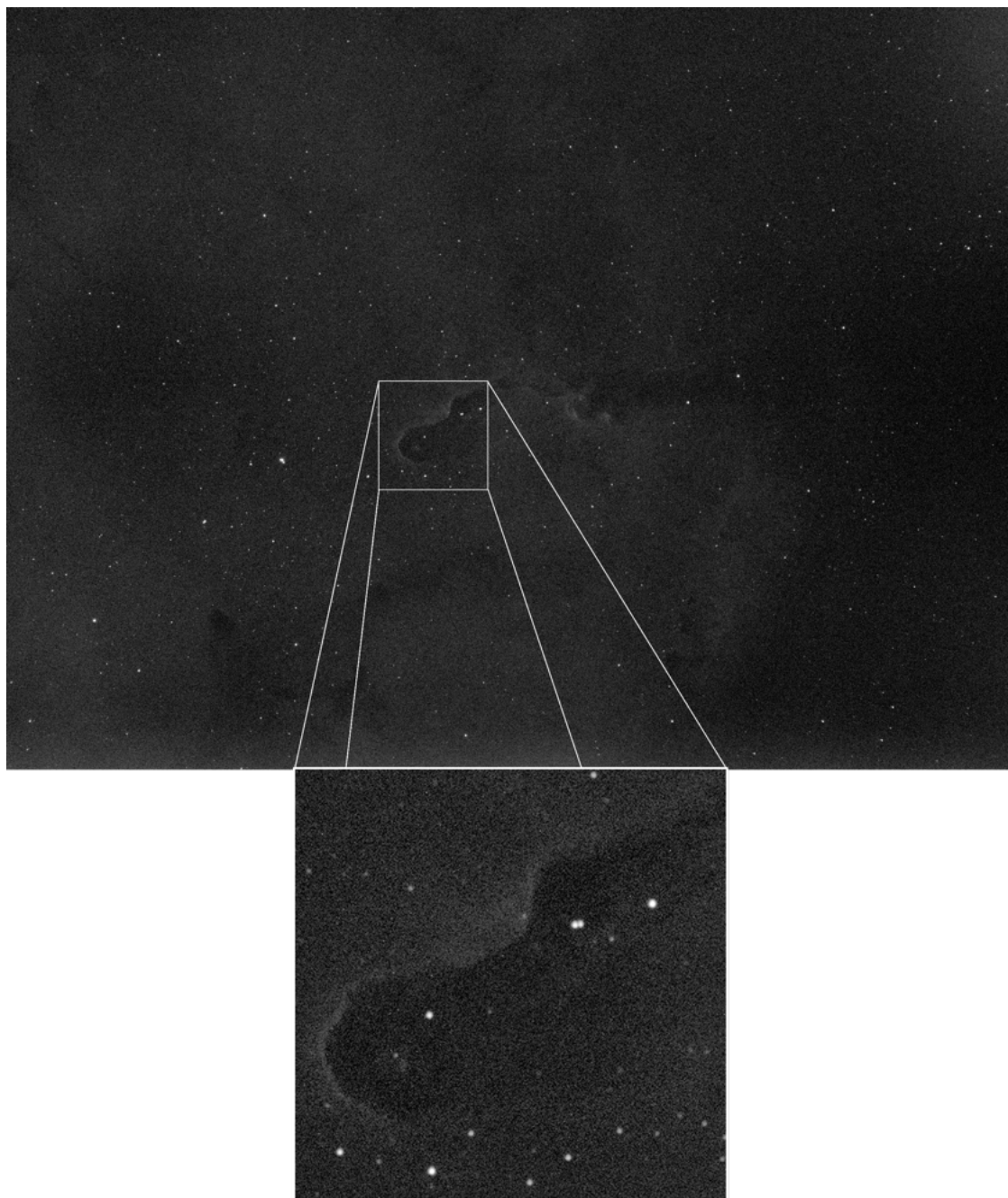
Slika 21: Maglica Veil

⁴HOO kombinacija je kombinacija gdje se povezuju zasebni RGB kanali sa crno bijelim slikama slikanim kroz Ha i OIII filter (R = Ha, G = OIII, B = OIII).

Slike 22 i 23 fotografirane su ekspozicijom duljine 300 s, vrijednost gain-a 200 i senzorom ohlađenim na -20°C . Na slici 22 moguće je vidjeti mali pomak u zvijezdama, uzrok tomu je bio nalet vjetra.



Slika 22: Maglica Pelikan



Slika 23: Maglica Slonovska surla

7 Zaključak

Praćenje samo jedne zvijezde nije u potpunosti savršeno rješenje. Noćima kada je atmosfera mirna praćenje funkcionira savršeno, ali u slučaju pojave turbulencija u atmosferi postoji mogućnost prividnog "plesanja" zvijezdi u vidnom polju kamere što bi dovelo do izduženih zvijezda na finalnoj fotografiji duge ekspozicije glavne kamere. Zbog toga bi bilo poželjno postići praćenje svih vidljivih zvijezda, no to zahtijeva potpunu preradu osnovne funkcionalnosti programa.

Kao što je prikazano na slikama u prethodnom poglavlju, program funkcionira s velikom točnošću u održavanju zvijezde na istom dijelu senzora kamere za praćenje i time se ostvaruju okrugle zvijezde na slikama dugih ekspozicija glavne kamere.

Literatura

- [1] *Ekspozicija*. Hrvatski jezični portal. URL: https://hjp.znanje.hr/index.php?show=search_by_id&id=fF1hXBk%253D&keyword=ekspozicija.
- [2] Vedran Vrhovac. *Tipovi Teleskopa*. URL: http://www.inet.hr/~vevrhova/xtra/teleskopi/teleskopi_2.html.
- [3] *Yerkes Observatory, Legacy*. Yerkes Observatory. URL: <https://www.yerkesobservatory.org/our-legacy/>.
- [4] *Skywatcher Mount AZ5 with Tripod*. Astroshop. URL: <https://www.astroshop.eu/alt-azimuth-without-goto/skywatcher-mount-az5-with-tripod/p,49495>.
- [5] *HyperTune® Service for the EQ6-R Mounts*. HyperTune. URL: <https://www.deepspaceproducts.com/product/hypertune-service-for-the-eq6-r-mounts>.
- [6] Drago Špoljarić. *Nebeski koordinatni sustavi*. URL: <https://eskola.zvjezdarnica.hr/osnove-astronomije/polozajna-i-efemeridna-astronomija/nebeski-koordinatni-sustavi/>.
- [7] mac01. *Autoguiding question to the forum*. URL: <https://www.dpreview.com/forums/thread/3880118>.
- [8] Linwood. *Thin transparent clouds: Keep or exclude from stack?* URL: <https://www.cloudynights.com/topic/725218-thin-transparent-clouds-keep-or-exclude-from-stack/>.
- [9] atan2. Wikipedia. URL: <https://en.wikipedia.org/wiki/Atan2>.
- [10] *Image Thresholding*. Open Source Computer Vision Library. URL: https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html.

Prilozi

Datoteka control.py

```
1 from distutils.log import debug
2 import cv2 as cv
3 import numpy as np
4 import camera
5 import time
6
7 class Guider:
8     stopGuiding = False
9     def __init__(self):
10         self.starX = 0
11         self.starY = 0
12         self.angle = 247
13         self.pulseStrenghtdir = np.array([0.5, 0.5, 0.45, 0.45], np.
14             float32)
15         self.calibrated = True
16         self.debug = False
17         self.dir = np.array(["N", "S", "E", "W"])
18         #self.angle = -100.2 % 360
19
20     def StarSelect(self, posX, posY):
21         image = cv.imread("./image_mono.tiff")
22
23         roi = image[posY - 16:posY + 16, posX - 16:posX + 16]
24
25         y1, x1 = self.StarCenter(roi)
26
27         posX += x1 - 16
28         posY += y1 - 16
29
30         roi = image[posY - 16:posY + 16, posX - 16:posX + 16]
31
32         if(y1 != -1 and x1 != -1):
33             self.starX = posX
34             self.starY = posY
35
36             print(self.starX)
37             print(self.starY)
38             return True
39         else:
40             return False
41
42     def RotateImage(self, image, angle, pointX = 0, pointY = 0):
43         if((pointX or pointY) == 0):
```

```
43         pointX = image.shape[1] / 2
44         pointY = image.shape[0] / 2
45
46         M = cv.getRotationMatrix2D((pointX, pointY), angle, 1)
47         return cv.warpAffine(image, M, (image.shape[1], image.shape[0]
48             ]))
49
50     def StarCenter(self, image):
51         y = image.shape[0]
52         x = image.shape[1]
53
54         center = np.array([0, 0], np.uint8)
55
56         blur = cv.GaussianBlur(image, (5, 5), 0)
57
58         maxBrightness = np.max(image)
59
60         ret, binarized = cv.threshold(blur, maxBrightness - 80, 255,
61             cv.THRESH_BINARY)
62
63         # Pocetne vrijednosti za izracun sredista zvijezde (najvece i
64             najmanje vrijednosti koordinata zvijezde)
65         maxX, maxY = 0, 0
66         minX, minY = x, y
67         n = 0
68
69         # i = x
70         # j = y
71         for i in range(1, x - 1):
72             for j in range(1, y - 1):
73                 # Looks if current pixel and pixels around it are
74                     brighter than 255
75                 if(binarized.item(j + 1, i, 1) >= 255 and binarized.
76                     item(j - 1, i, 1) >= 255 and binarized.item(j, i +
77                         1, 1) >= 255 and binarized.item(j, i - 1, 1) >= 2
78                         55 and
79                 binarized.item(j, i, 1) >= 255):
80                     if(j > maxY):
81                         maxY = j
82                     if(i > maxX):
83                         maxX = i
84                     if(j < minY):
85                         minY = j
86                     if(i < minX):
87                         minX = i
88
89                 n += 1
```

```

83
84     if(n >= 3 and n < x * y - x * y * 0.3 and np.min(binarized) ==
85         0):
86         # Izracun koordinata centra zvijezde
87         center = int((minY + maxY) / 2), int((minX + maxX) / 2)
88
89         if(debug):
90             print(center)
91
92         return center
93
94     else:
95         print("No stars found")
96         return -1, -1
97
98 def CalibrateAngle(self):
99     angle1, angle2, = 0, 50
100
101     while(np.abs(angle1 - angle2) > 30):
102         for i in range(0, 4, 2):
103             camera.TimedGuide(self.dir[i], 5)
104
105             image = cv.imread(camera.capture(1, 75))
106
107             roi = image[self.starY - 16:self.starY + 16, self.
108                 starX - 16:self.starX + 16]
109
110             y2, x2 = self.StarCenter(roi)
111
112             #Calculate the angle between right center/top center
113             #and new star location
114             #(3): https://stackoverflow.com/questions/1211212/how-
115             #to-calculate-an-angle-from-three-points
116             if(i == 0):
117                 angle1 = 360 - np.abs(np.rad2deg(np.arctan2((x2 -
118                     16) * -1, (y2 - 16) * -1)))
119             if(i == 2):
120                 angle2 = 360 - np.abs(np.rad2deg(np.arctan2(y2 - 1
121                     6, x2 - 16)))
122
123             # For debugging
124             if(debug):
125                 print(self.dir[i])
126                 print(angle1)
127                 print(angle2)
128                 print("\tX: ", x2)
129                 print("\tY: ", y2)

```

```
124
125         print("Moving back")
126         camera.TimedGuide(self.dir[i + 1], 5)
127
128         # capture new image
129         image = cv.imread(camera.capture(1, 75))
130
131         roi = image[self.starY - 16:self.starY + 16, self.
132                 starX - 16:self.starX + 16]
133         # find star center coordinates
134         y2, x2 = self.StarCenter(roi)
135
136         if(x2 == -1 or y2 == -1):
137             print("Calibration failed\nLost guide star")
138             i = 0
139
140         self.starX += x2 - 16
141         self.starY += y2 - 16
142
143         if(np.abs(angle1 - angle2) > 30 and i == 2):
144             print("Angle calibration failed, rerunning")
145
146         print("Angle calibration success")
147
148         angle = (angle1 + angle2) / 2
149
150         self.angle = angle
151
152     def CalibrateStrenght(self):
153         while(all([pulse >= 1.7 for pulse in self.pulseStrenghtdir])):
154             for i in range(4):
155                 # move north, south, east, west
156                 print("Calculating pulse strenght: " + self.dir[i])
157
158                 camera.TimedGuide(self.dir[i], 5)
159
160                 image = cv.imread(camera.capture(1, 75))
161                 xOld = self.starX
162                 yOld = self.starY
163
164                 roi = image[self.starY - 16:self.starY + 16, self.
165                         starX - 16:self.starX + 16]
166                 self.starX += self.StarCenter(roi)[1] - 16
167                 self.starY += self.StarCenter(roi)[0] - 16
168
169                 image = self.RotateImage(image, self.angle, xOld, yOld
170 )
```

```

168         roi = image[yOld - 16:yOld + 16, xOld - 16:xOld + 16]
169
170         dirY, dirX = self.StarCenter(roi)
171
172         if(i < 2):
173             if(dirY != 16):
174                 self.pulseStrenghtdir[i] = np.abs(5 / (dirY -
175                                     16))
176             else:
177                 if(dirX != 16):
178                     self.pulseStrenghtdir[i] = np.abs(5 / (dirX -
179                                     16))
180
181             print("self.direction " + self.dir[i] + ": " + str(
182                 self.pulseStrenghtdir[i]))
183
184     def Calibrate(self, image=cv.imread("./image_mono.tiff")):
185         self.stopGuiding = False
186         if(not self.calibrated):
187             self.CalibrateAngle()
188             self.CalibrateStrenght()
189             self.calibrated = True
190
191         return self.RotateImage(image, self.angle, self.starX, self.
192             starY)
193
194     def Guide(self, expouser = 1, gain = 75):
195         print("Guiding")
196         maxDir = "0"
197         minDir = "0"
198
199         while not self.stopGuiding:
200             image = cv.imread(camera.capture(expouser, gain))
201             time.sleep(expouser+0.1)
202
203             image = self.Calibrate(image)
204             roi = image[self.starY - 16:self.starY + 16, self.starX -
205                 16:self.starX + 16]
206             cv.imwrite("./roi.tiff", roi)
207
208             self.dirY, self.dirX = self.StarCenter(roi)
209
210             if(self.dirY == -1):
211                 camera.StopGuide("N")
212                 camera.StopGuide("S")
213                 camera.StopGuide("E")

```

```
210         camera.StopGuide("W")
211
212     else:
213         self.dirX -= 16
214         self.dirY -= 16
215
216         if(self.dirX > 0):
217             maxTime = self.pulseStrenghtdir[2] * np.abs(self.
                dirX)
218
219             maxDir = "W"
220         elif(self.dirX < 0):
221             maxTime = self.pulseStrenghtdir[3] * np.abs(self.
                dirX)
222
223             maxDir = "E"
224         else:
225             maxTime = 0
226
227         if(self.dirY > 0):
228             minTime = self.pulseStrenghtdir[0] * np.abs(self.
                dirY)
229
230             minDir = "N"
231         elif(self.dirY < 0):
232             minTime = self.pulseStrenghtdir[1] * np.abs(self.
                dirY)
233
234             minDir = "S"
235         else:
236             minTime = 0
237
238         if(minTime > maxTime):
239             temp = maxTime
240             maxTime = minTime
241             minTime = temp
242
243             temp = maxDir
244             maxDir = minDir
245             minDir = temp
246
247         starTime = time.time()
248         camera.StartGuide(maxDir)
249         camera.StartGuide(minDir)
250
251         if(maxTime > 2): maxTime = 2
252         if(minTime > 2): minTime = 2
```

```
253
254         maxTime = maxTime - minTime
255
256         print(maxDir + "\t" + str(maxTime))
257         print(minDir + "\t" + str(minTime))
258
259         time.sleep(minTime)
260         camera.StopGuide(minDir)
261
262         time.sleep(maxTime)
263         camera.StopGuide(maxDir)
264
265         # if(maxTime != 0):
266         #     while(time.time() <= starTime + maxTime):
267         #         if(time.time() >= starTime + minTime):
268         #             camera.StopGuide(minDir)
269         #         camera.StopGuide(maxDir)
```

Datoteka camera.py

```
1 import zwoasi as asi
2 import time
3
4 global binning
5 binning = 1
6
7 asi.init('./1.18/linux_sdk/lib/armv7/libASICamera2.so.1.18')
8
9 num_cameras = asi.get_num_cameras()
10 if num_cameras == 0:
11     raise ValueError('No cameras found')
12
13 camera_id = 0 # use first camera from list
14 cameras_found = asi.list_cameras()
15 print(cameras_found)
16 camera = asi.Camera(camera_id)
17
18 # Use minimum USB bandwidth permitted
19 camera.set_control_value(asi.ASI_BANDWIDTHOVERLOAD, camera.
    get_controls()['BandWidth']['MinValue'])
20
21 camera.set_control_value(asi.ASI_MONO_BIN, binning)
22 camera.set_roi(bins=binning)
23
24 # Set some sensible defaults. They will need adjusting depending upon
25 # the sensitivity, lens and lighting conditions used.
26 camera.disable_dark_subtract()
27
28 def capture(expouser, gain):
29     while(True):
30         camera.set_control_value(asi.ASI_GAIN, gain)
31         camera.set_control_value(asi.ASI_EXPOSURE, int(expouser * 1000
            000)) # microseconds
32
33         filename = 'image_mono.tiff'
34         camera.set_image_type(asi.ASI_IMG_RAW8)
35         if(camera.capture(filename=filename)):
36             break
37
38     return "./image_mono.tiff"
39
40 def TimedGuide(direction, duration):
41     if direction == "N":
42         camera.pulse_guide_on(asi.ASI_GUIDE_NORTH)
43     elif direction == "S":
```



```
44     camera.pulse_guide_on(asi.ASI_GUIDE_SOUTH)
45 elif direction == "E":
46     camera.pulse_guide_on(asi.ASI_GUIDE_EAST)
47 elif direction == "W":
48     camera.pulse_guide_on(asi.ASI_GUIDE_WEST)
49
50 print("Moving: " + direction)
51
52 time.sleep(duration)
53
54 camera.pulse_guide_off(asi.ASI_GUIDE_NORTH)
55 camera.pulse_guide_off(asi.ASI_GUIDE_SOUTH)
56 camera.pulse_guide_off(asi.ASI_GUIDE_EAST)
57 camera.pulse_guide_off(asi.ASI_GUIDE_WEST)
58
59 time.sleep(duration)
60
61 def StartGuide(direction):
62     if direction == "N":
63         camera.pulse_guide_on(asi.ASI_GUIDE_NORTH)
64     elif direction == "S":
65         camera.pulse_guide_on(asi.ASI_GUIDE_SOUTH)
66     elif direction == "E":
67         camera.pulse_guide_on(asi.ASI_GUIDE_EAST)
68     elif direction == "W":
69         camera.pulse_guide_on(asi.ASI_GUIDE_WEST)
70
71 def StopGuide(direction):
72     if direction == "N":
73         camera.pulse_guide_off(asi.ASI_GUIDE_NORTH)
74     elif direction == "S":
75         camera.pulse_guide_off(asi.ASI_GUIDE_SOUTH)
76     elif direction == "E":
77         camera.pulse_guide_off(asi.ASI_GUIDE_EAST)
78     elif direction == "W":
79         camera.pulse_guide_off(asi.ASI_GUIDE_WEST)
```

Datoteka gui.py

```
1 import tkinter as tk
2 from tkinter import *
3 from xml.etree.ElementPath import get_parent_map
4 from PIL import Image, ImageTk
5 import control
6 from os import execl, path
7 from sys import executable, argv
8 import camera
9 import threading
10 from time import sleep, ctime, time
11 import os
12
13 root = Tk()
14 root.geometry("400x400")
15 root.title("Guider")
16
17 class Gui:
18     def __init__(self, root):
19         self.starSelectBool = False
20         self.guiders = control.Guiders()
21
22         self.canvas = tk.Canvas(root, height=1500, width=1500)
23         self.canvas.pack()
24
25         self.frame = tk.Frame(root, bg="black")
26         self.frame.place(rely = 0.8, relwidth = 1, relheight=1)
27
28         image = Image.open('./image_mono.tiff')
29         # test.image = image
30         # If image is larger than 1000 pixels downsample it by a
31         # factor of 2
32         self.downsample = 1
33         if(image.height >= 1000 or image.width >= 1000):
34             image = image.resize((int(image.width / 2), int(image.
35                 height / 2)), Image.NEAREST)
36             self.downsample = 2
37
38         # Make it readable by tkinter
39         img = ImageTk.PhotoImage(image)
40
41         # Place the image using label widget
42         self.imagePreview = Label(self.canvas, image = img, bg="black")
43         self.imagePreview.pack()
```

```

43     # Buttons to:
44     #   to select a star
45     self.selectStarButton = tk.Button(root, text="Select a Star",
46                                       highlightbackground = "blue", highlightcolor= "black",
47                                       command=self.SelectStar)
46     self.selectStarButton.place(relx=0, rely=0.8, width=150)
47     #   start guiding
48     self.startGuidingButton = tk.Button(root, text="Start Guiding"
49                                       , highlightbackground = "green", highlightcolor= "green",
50                                       command=threading.Thread(target=self.StartGuiding).start)
49     self.startGuidingButton.place(relx=0, rely=0.85, width=150)
50     #   stop guiding
51     self.stopGuidingButton = tk.Button(root, text="Stop Guiding",
52                                       highlightbackground = "red", highlightcolor= "black",
53                                       command=self.StopGuiding)
52     self.stopGuidingButton.place(relx=0, rely=0.9, width=150)
53     #   to restart the app
54     self.restartButton = tk.Button(root, text="R",
55                                       highlightbackground = "black", highlightcolor= "black",
56                                       command=self.Restart)
55     self.restartButton.place(relx=0.95, rely=0.95)
56
57     # Bottom console showing current status
58     self.text = Text(root, bg="black", highlightbackground = "
59         black", highlightcolor= "black")
59     self.text.insert(INSERT, "Status: looping images")
60     self.text.config(state=DISABLED, fg="white")
61     self.text.place(relx=0, rely=0.95, width=250)
62
63     # exposure input
64     self.exposureInputLabel = Label(root, text="Exposure:", bg="
65         black", fg="white")
65     self.exposureInputLabel.place(relx=0.24, rely=0.8)
66     self.exposureInput = Entry(root, fg="white", bg="black",
67                               highlightbackground = "black", highlightcolor= "black")
67     self.exposureInput.insert(0, '1.00')
68     self.exposureInput.place(relx=0.35, rely=0.8)
69
70     # Gain input
71     self.gainInputLabel = Label(root, text="Gain:", bg="black", fg
72                               ="white")
72     self.gainInputLabel.place(relx=0.285, rely=0.85)
73     self.gainInput = Entry(root, fg="white", bg="black",
74                               highlightbackground = "black", highlightcolor= "black")
74     self.gainInput.insert(0, '75')
75     self.gainInput.place(relx=0.35, rely=0.85)
76

```

```
77     # Setting exposure and gain
78     self.exposure = float(1.0)
79     self.gain = 75
80
81     root.bind('<Return>', self.TextInput)
82
83     root.wm_minsize(image.width + 2, image.height + 120)
84
85     threading.Thread(target=self.DisplayImage).start()
86
87     root.mainloop()
88
89     def TextInput(self, event):
90         self.exposure = float(self.exposureInput.get())
91         self.gain = int(self.gainInput.get())
92
93     def SelectStar(self):
94         self.ConsoleInput("Status: Selecting a star", "white")
95
96         # Change the mouse cursor when selecting a star
97         if(not self.starSelectBool):
98             self.imagePreview.bind("<Button-1>", self.ClickEvent)
99             self.imagePreview.configure(cursor="icon")
100         else:
101             self.imagePreview.unbind("<Button-1>")
102             self.imagePreview.configure(cursor="arrow")
103
104         self.starSelectBool = not self.starSelectBool
105
106     def ClickEvent(self, event):
107         # Check if a star has been found in the selected roi
108         if(self guider.StarSelect(event.x * self.downsample, event.y *
109             self.downsample)):
110             self.ConsoleInput("Status: Found a star", "#34D321")
111
112             # Disable clickable label
113             self.imagePreview.unbind("<Button-1>")
114             # Change back to og mouse cursor
115             self.imagePreview.configure(cursor="arrow")
116
117             self.starSelectBool = not self.starSelectBool
118         else:
119             self.ConsoleInput("Status: No stars found", "red")
120
121     def StartGuiding(self):
122         self.ConsoleInput("Status: Calibrating", "#34D321")
```

```
123         self.guider.Calibrate()
124
125         self.ConsoleInput("Status: Guiding", "#34D321")
126         self.guider.Guide(self.exposure, self.gain)
127
128     def StopGuiding(self):
129         self.ConsoleInput("Status: Guiding stopped", "red")
130         self.guider.stopGuiding = True
131
132     def Restart(self):
133         camera.StopGuide("N")
134         camera.StopGuide("S")
135         camera.StopGuide("E")
136         camera.StopGuide("W")
137
138         self.python = executable
139         execl(self.python, self.python, * argv)
140
141     def ConsoleInput(self, txt, fontColor):
142         self.text.config(state=NORMAL, fg=fontColor)
143         self.text.delete('1.0', END)
144         self.text.insert(INSERT, txt)
145         self.text.config(state=DISABLED)
146
147     def DisplayImage(self):
148         while(True):
149             if(os.path.getsize('image_mono.tiff') > 1228000):
150                 image = Image.open('./image_mono.tiff')
151                 image = image.resize((int(image.width / 2), int(image.
152                                     height / 2)), Image.NEAREST)
153
154                 # Make it readable by tkinter
155                 img = ImageTk.PhotoImage(image)
156
157                 # Place the image using label widget
158                 self.imagePreview.config(image = img)
159                 # Solves blinking, https://stackoverflow.com/questions
160                 # /48364168/flickering-video-in-opencv-tkinter-
161                 # integration
162                 self.imagePreview.image = img
163
164             else:
165                 print("ERROR")
166
167             # If guiding isn't active loop images
168             if self.guider.stopGuiding:
169                 camera.capture(self.exposure, self.gain)
```

```
167         else:
168             # Wait until the exposure is taken
169             sleep(self.exposure)
170
171 if __name__ == "__main__":
172     camera.capture(1, 75)
173     giu = Gui(root)
```